Agregación de modelos

Métodos “Ensemble”

Métodos de ensemble

- La idea es combinar varios modelos “simples” para obtener un mejor resultado agregado.
- Podemos denominar a estos métodos como “meta-algoritmos”.
- Por ejemplo, supongamos que tenemos tres regresores/clasificadores, $h_1(y)$, $h_2(y)$ y $h_3(y)$, cada uno de ellos posiblemente elaborado por métodos independientes e incluso usando características no comunes.
- Dado un punto de test, ¿cómo determinamos a qué clase pertenece o cuál es el valor de la función objetivo?
  - La media de las estimaciones (regresión) o la clase más frecuente (clasificación) puede ser una buena opción inicial.

Bagging (Bootstrap Aggregation)

- La idea del bagging es promediar el resultado de varios clasificadores generados a partir de los datos.
- Se puede generar un modelo de Bagging mediante el siguiente algoritmo (se entrenan por independiente con datos aleatorios por reemplazo):

La idea es que con este procedimiento conseguiremos estimadores más robustos.

Algoritmo 1 Bagging

1: Sea M el número de predictores requeridos.
2: $d = \{(x_1, y_1), \ldots, (x_N, y_N)\}$
3: **for** $i = 1$ hasta M **do**
4: Generar una nueva muestra d_{bag}, eligiendo N muestras desde d con reemplazo.
5: Entrenar un estimador f_i con la muestra d_{bag} y agregarlo al ensemble.
6: **end for**
7: Para cada patrón de test x, la salida esta dada por: $f_{bag}(x) = \frac{1}{M}\sum_i f_i(x)$

Boosting

- Busca combinar K “weak classifiers” para obtener un clasificador más robusto.
- En lugar de dar la misma importancia a todos las muestras para el ajuste de todos los clasificadores como en bagging, pondera las muestras para el entrenamiento dando más importancia a unas que a otros a la hora de ajustar un clasificador.
- Un clasificador débil es aquel cuyos resultados son simplemente mejores que una clasificación al azar (es decir, porcentaje de acierto ligeramente superior a 1/nc).
- Una implementación muy popular es el algoritmo “AdaBoost”.

AdaBoost

Algorithm 1 (AdaBoost)

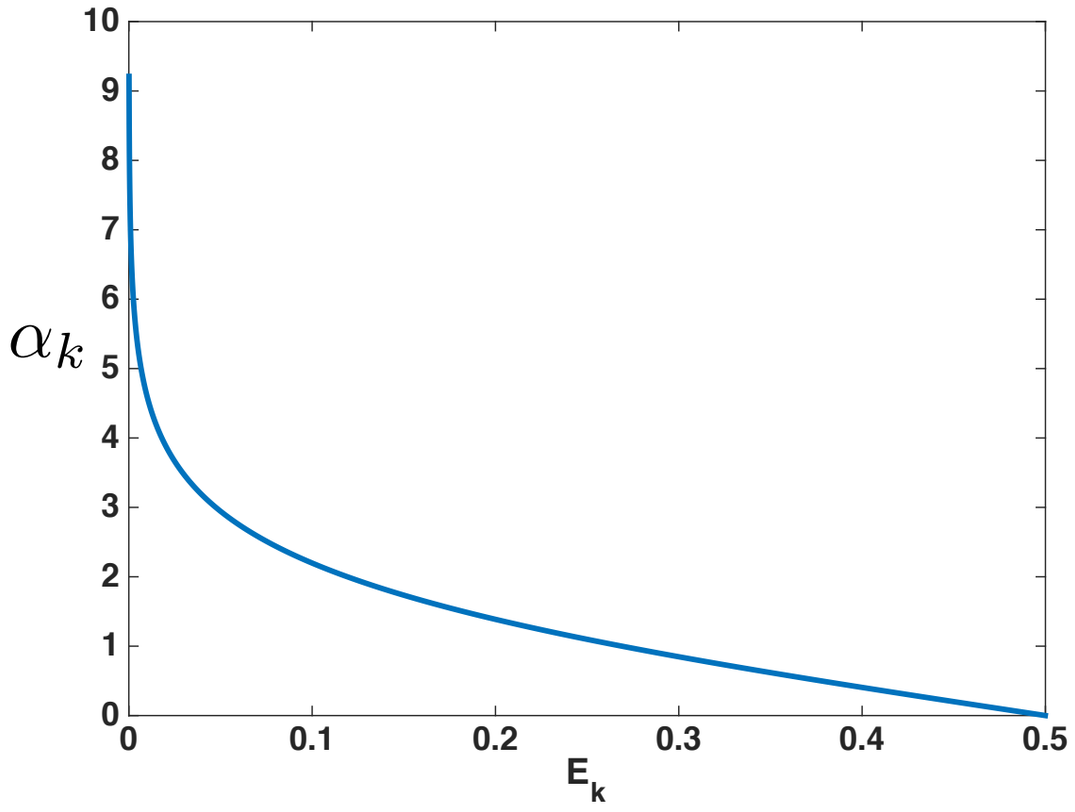
begin **initialize** $\mathcal{D} = \{\mathbf{x}^1, y_1, \mathbf{x}^2, y_2, \ldots, \mathbf{x}^n, y_n\}, k_{max}, W_1(i) = 1/n, i = 1, \ldots, n$
  $k \leftarrow 0$
  do $k \leftarrow k + 1$
    Train weak learner C_k using $\mathcal{D}$ sampled according to distribution $W_k(i)$
    $E_k \leftarrow$ Training error of C_k measured on $\mathcal{D}$ using $W_k(i)$
    $\alpha_k \leftarrow \frac{1}{2}\ln[(1 - E_k)/E_k]$
    $W_{k+1}(i) \leftarrow \frac{W_k(i)}{Z_k} \times \begin{cases} e^{-\alpha_k} & \text{if } h_k(\mathbf{x}^i) = y_i \text{ (correctly classified)} \\ e^{\alpha_k} & \text{if } h_k(\mathbf{x}^i) \neq y_i \text{ (incorrectly classified)} \end{cases}$
  until $k = k_{max}$
return C_k and α_k for $k = 1$ to k_{max} (ensemble of classifiers with weights)
end

The final classification decision of a test point $\mathbf{x}$ is based on a discriminant function that is merely the weighted sums of the outputs given by the component classifiers:

$$g(\mathbf{x}) = \left[\sum_{k=1}^{k_{max}} \alpha_k h_k(\mathbf{x})\right]. \tag{36}$$

Zk es un factor de normalización

AdaBoost



α_k determina la importancia del clasificador k en el ensemble en función de su error de clasificación

AdaBoost

In this way, AdaBoost "focuses in" on the informative or "difficult" patterns. Specifically, we initialize these weights across the training set to to be uniform. On each iteration k, we draw a training set at random according to these weights, and train component classifier C_k on the patterns selected. Next we increase weights of training patterns misclassified by C_k and decrease weights of the patterns correctly classified by C_k. Patterns chosen according to this new distribution are used to train the next classifier, C_{k+1}, and the process is iterated.

The final classification decision of a test point $\mathbf{x}$ is based on a discriminant function that is merely the weighted sums of the outputs given by the component classifiers:

$$g(\mathbf{x}) = \left[\sum_{k=1}^{k_{max}} \alpha_k h_k(\mathbf{x}) \right]. \tag{36}$$

Random Forests

- Método de bagging aplicado a la generación de árboles de clasificación con selección aleatoria de nodos.
- Intenta evitar problemas de sobreentrenamiento en árboles de decisión.
- En el proceso de entrenamiento, a la hora de determinar la división de un nodo no se consideran todas las posibles divisiones (características), sino un número mucho menor de las disponibles (seleccionadas aleatoriamente de entre las disponibles). Es en consecuencia necesario que el número de características sea elevado.
- Se genera un número alto de árboles de decisión mediante este procedimiento.
- La decisión final sobre un punto de test se basa en el promediado o consideración conjunta de las decisiones particulares de cada árbol.

Cascada de clasificadores

- El reconocimiento de objetos es factible mediante un único clasificador global.
- No obstante, en ocasiones es útil recurrir a una cascada de clasificadores fuertes, cuya aportación fundamental puede ser reducir el tiempo total de procesamiento así como facilitar el entrenamiento de los clasificadores.
- En cada capa se realiza una preclasificación de los objetos, no considerándose en etapas posteriores las clases descartadas.
- Las clases objetivo en etapas intermedias no tienen por qué coincidir con las clases objetivo, sino que pueden ser superclases de las mismas (por ejemplo, objetos cuadrados o circulares).